



ESCUELA DE
INGENIERÍA EN CIENCIAS Y SISTEMAS
FACULTAD DE INGENIERÍA
UNIVERSIDAD DE SAN CARLOS DE GUATEMALA



Fecha:

DD/MM/2026

Analisis y Diseño de Sistemas 1

Escuela de Ingeniería de Ciencias Y Sistemas

NOMBRE DEL TUTOR

UNIDAD 1

Control de Versiones

Anuncios Importantes



Foro Semanal



Asignacion DTT



Anuncio 3



Anuncio 4



Anuncio 5

Agenda



TEMA 1



TEMA 2



TEMA 3



TEMA 4



TEMA 5

COMPETENCIA(S) QUE DESARROLLAREMOS



Genera soluciones de software aplicando buenas prácticas de análisis, control de versiones y metodologías ágiles para garantizar proyectos eficientes y colaborativos en entornos virtuales de trabajo

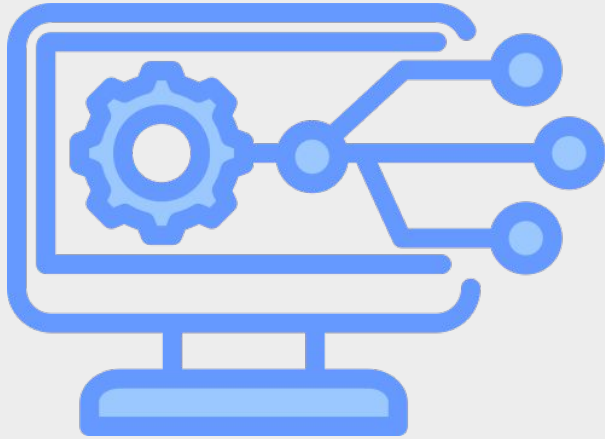


RESPECTO

"Imagina que trabajas en equipo y uno de tus compañeros pasó horas corrigiendo errores y mejorando una parte del código. Si tú llegas y subes una versión sin revisar los cambios previos, o sobrescribes su trabajo sin avisar, estás faltando al respeto a su esfuerzo y al trabajo en equipo.

Usar bien un sistema de control de versiones no solo es cuestión técnica, también es una forma de mostrar respeto: respetas el tiempo de los demás, su conocimiento, y la colaboración. Usar 'pull requests', comentar tus cambios, revisar lo que otros hicieron antes de modificar, son actos de cortesía profesional que demuestran respeto."

SISTEMAS DE CONTROL DE VERSIONES



Son un tipo de software diseñado para realizar un seguimiento de las modificaciones efectuadas en el código a lo largo del tiempo.

Cuando un desarrollador edita el código, el sistema de control de versiones captura instantáneas de los archivos, almacenándolas de manera permanente para posibilitar su recuperación en el futuro, si así se requiere.

ANTECEDENTES

1

El Problema Inicial

En los primeros días del desarrollo de software, era común perder o sobrescribir código, especialmente en equipos grandes. Los desarrolladores necesitaban una forma de guardar y gestionar cambios de manera organizada.

2

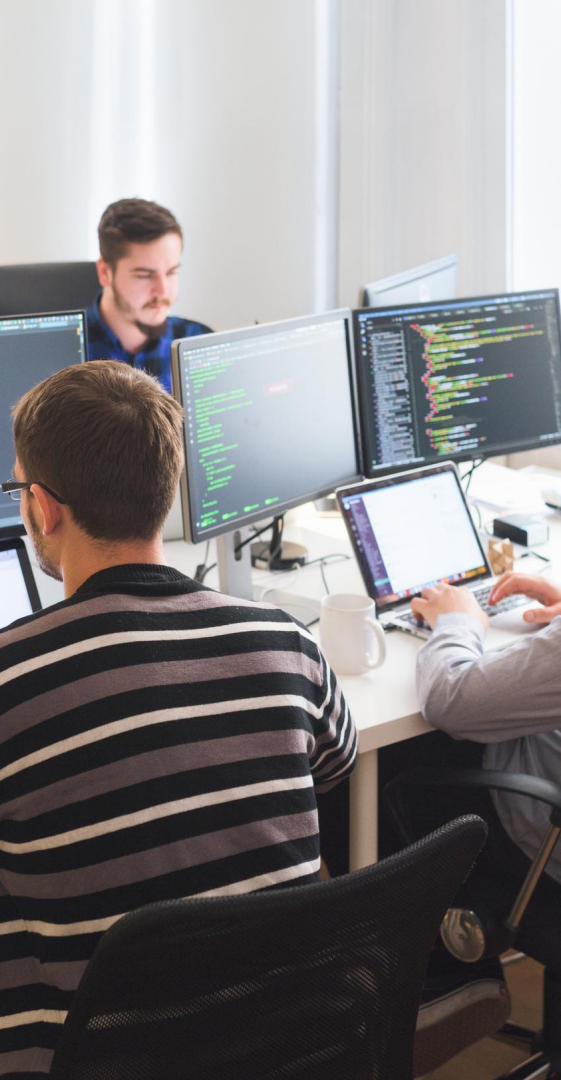
Primera Generación: Sistemas Centralizados (1970s)

Surgieron sistemas como SCCS y RCS que permitieron almacenar versiones de código de forma centralizada. Sin embargo, tenían limitaciones para el trabajo en equipo, ya que solo permitían a una persona editar un archivo a la vez.

3

Revolución con Sistemas Distribuidos (2000s)

Con la llegada de Git y Mercurial, cada desarrollador pudo tener una copia completa del proyecto, facilitando el trabajo paralelo y la integración de cambios. Esto revolucionó la colaboración, especialmente en proyectos de código abierto.



IMPORTANCIA DE LOS SISTEMAS DE CONTROL DE VERSIONES

1

Colaboración

Permite a los desarrolladores trabajar simultáneamente en el mismo proyecto sin interferir con el trabajo de los demás.

2

Seguimiento de cambios

Permite a los desarrolladores trabajar simultáneamente en el mismo proyecto sin interferir con el trabajo de los demás.

3

Restauración de versiones

En caso de error o de que se desee volver a una versión anterior del código, se puede restaurar la versión deseada de forma rápida y segura.



¿Cómo ha transformado la gestión de versiones de software?

Antes

- Dificultad para colaborar en proyectos.
- Alto riesgo de perder cambios o de que se sobrescriban.
- Dificultad para gestionar versiones del software.

Después

- Colaboración fluida y eficiente.
- Control total sobre los cambios del código y posibilidad de restaurar versiones anteriores.
- Gestión de versiones de software fácil y transparente.

TIPOS DE CONTROL DE VERSIONES



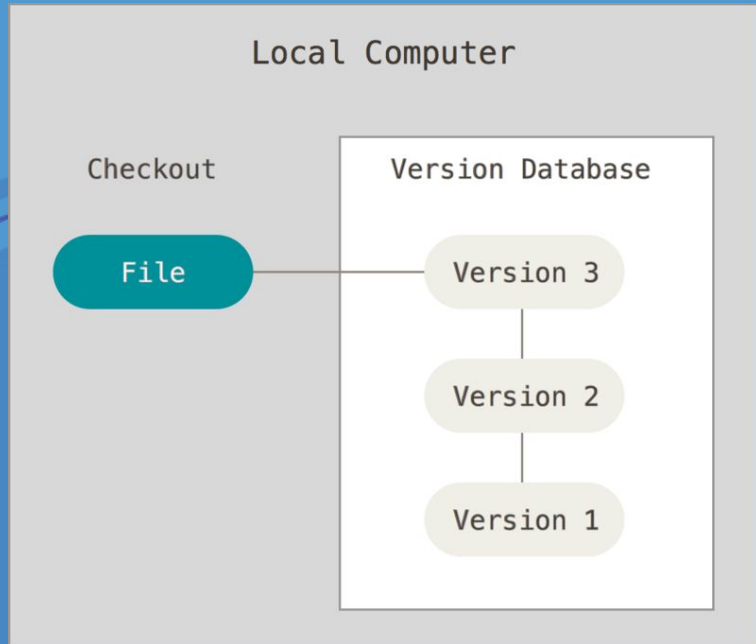
The diagram consists of three identical circular elements arranged horizontally. Each element features a thick blue outer ring with a lighter blue inner ring. The text is centered within the white space of each circle. The background is a light gray with abstract blue and white shapes on the left and right sides.

Locales

Centralizados

Distribuidos

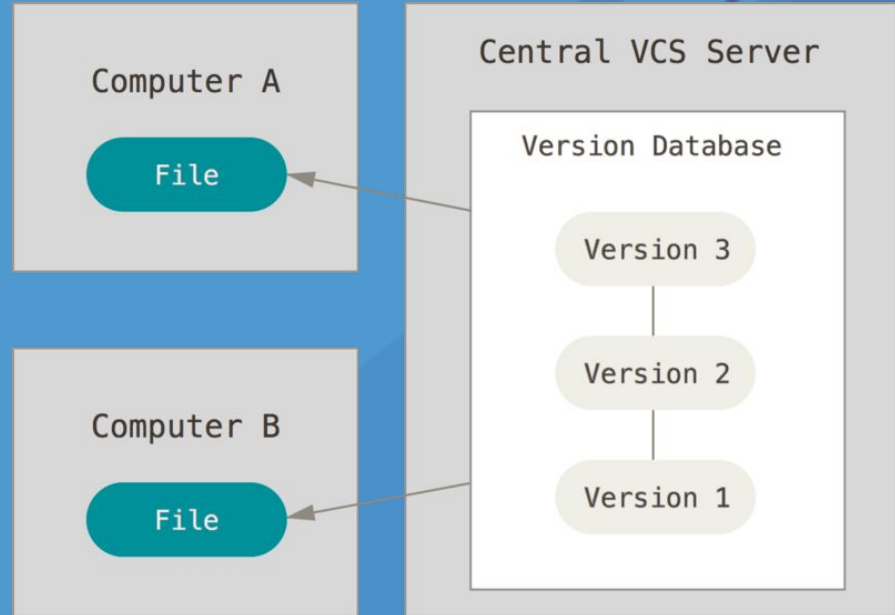
Sistemas de Control de Versiones Local



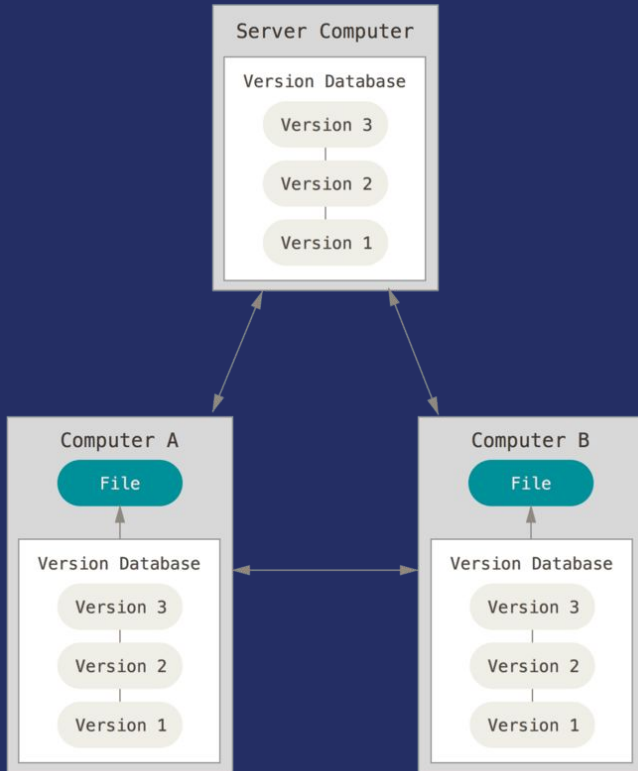
En este sistema las instantáneas o cambios en los archivos se registran y almacenan localmente en la máquina del usuario. Este enfoque puede ser adecuado para proyectos más pequeños o individuales donde la colaboración en línea no es esencial

Sistemas de Control de Versiones Centralizado

Estos sistemas cuentan con un repositorio central que almacena todas las versiones y cambios. En este modelo, cada usuario trabaja con su propia copia local de los archivos, pero las actualizaciones y cambios se registran en un servidor central.



Sistemas de Control de Versiones Distribuido



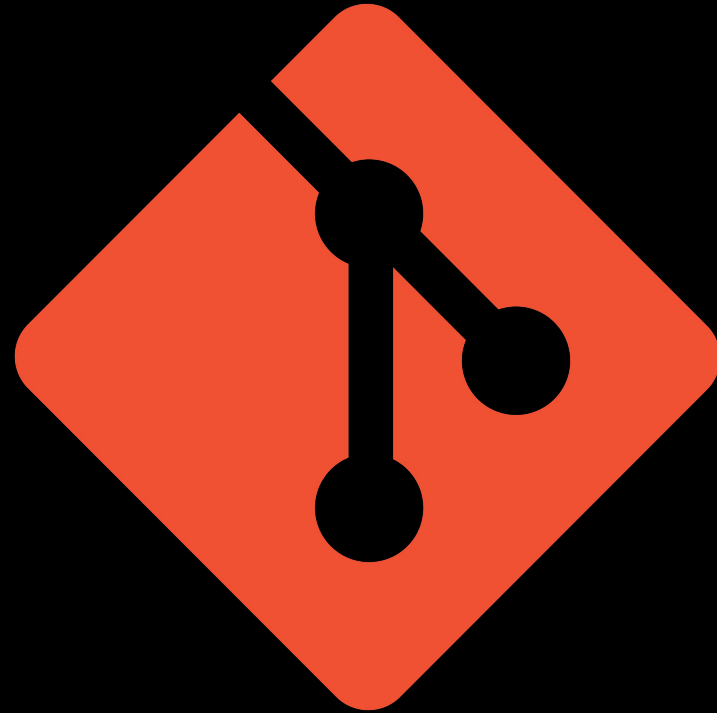
Los desarrolladores pueden trabajar de manera independiente en sus copias locales, realizar cambios, y después compartir esos cambios con otros usuarios a través de la red. Esto permite una mayor flexibilidad y autonomía en el desarrollo, ya que cada copia local es una entidad autónoma con su historial completo de versiones.

Introducción a Git



Git es el sistema de control de versiones moderno más utilizado del mundo, Fue creado por Linus Torvalds en 2005 como un proyecto de código abierto.

Presenta una arquitectura distribuida, siendo un ejemplo de DVCS (sistema de control de versiones distribuido).



Características

- Distribuido y Descentralizado
- Rastreo preciso de cambios
- Ramificación Eficiente
- Fusiones Simplificadas



¿Cuál es la diferencia entre git y GitHub?

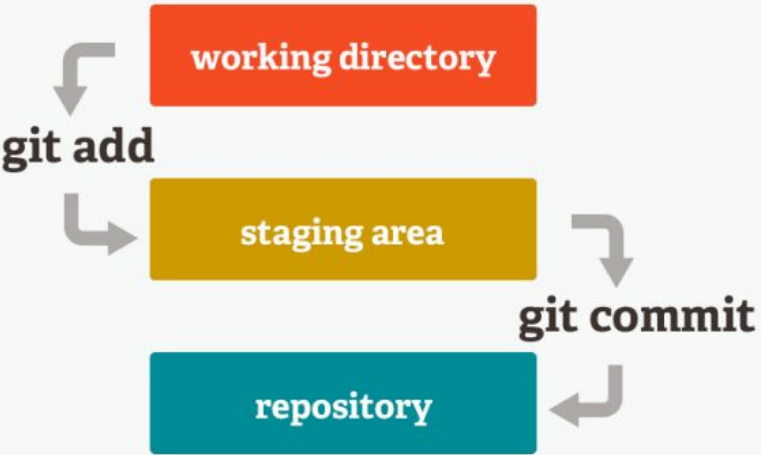
Git es el sistema de control de versiones en sí mismo, mientras que GitHub es una plataforma basada en la web que utiliza Git para proporcionar servicios adicionales relacionados con el desarrollo de software en equipo.



En conjunto, Git y GitHub forman una combinación poderosa para el desarrollo de software, permitiendo a los desarrolladores gestionar cambios localmente con Git y colaborar a escala global mediante GitHub.

Fases de Git

Las "fases de Git" se refieren a los diferentes estados por los que pasan los archivos en un repositorio Git durante el proceso de seguimiento de cambios y la preparación para realizar un commit.





Working Directory

Es la fase inicial donde se encuentran los archivos modificados o creados en un directorio. En esta etapa, los cambios no han sido aún registrados en el repositorio.



Staging Area

También conocida como "index", esta fase actúa como un espacio intermedio entre el directorio de trabajo y el repositorio. Aquí se seleccionan los cambios específicos que se desean incluir en el próximo commit.



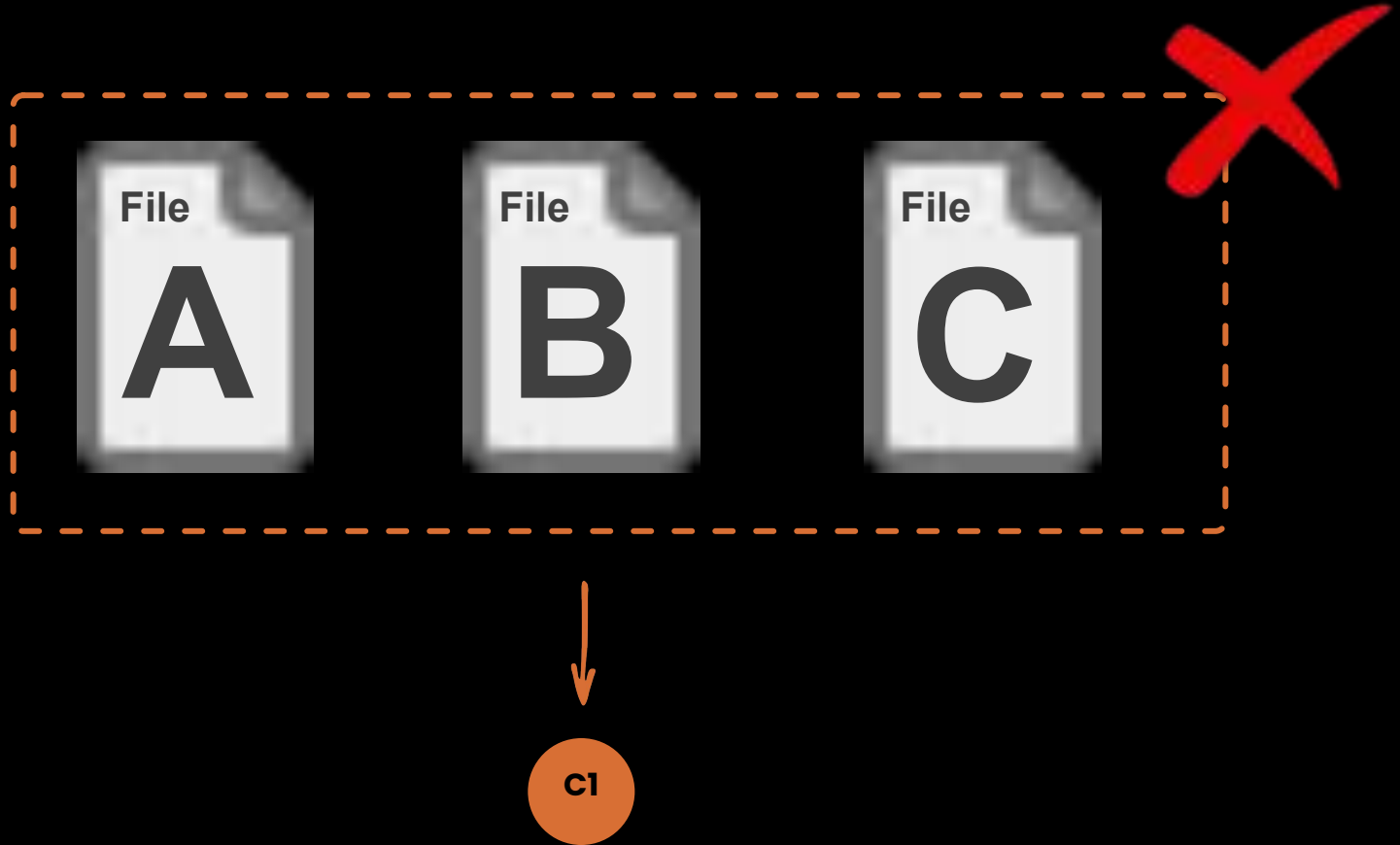
Git Repository

Es la fase final donde los cambios son confirmados permanentemente en la historia del repositorio. Una vez que los cambios se han añadido al repositorio, se convierten en parte de la historia del proyecto,

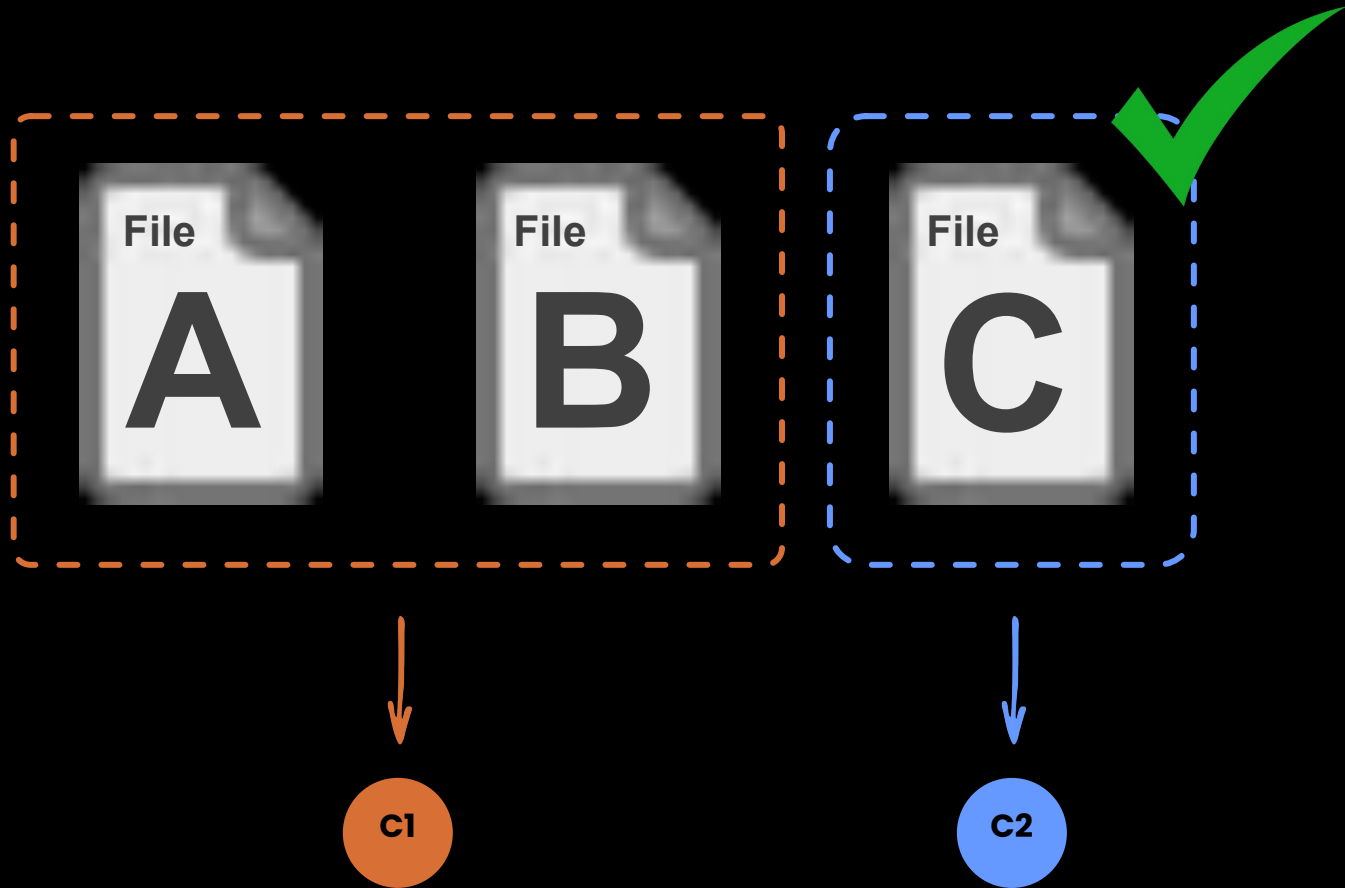
- Verificar estado del repositorio.
- Actualizar repositorio antes de hacer cambios.
- Desarrollar buenos commits.
- Evitar cambios directos en la rama main.
- Realizar un buen .gitignore.
- No colocar información sensible.

Buenas prácticas en git

Se debe evitar meter muchos cambios no relacionados en un mismo commit

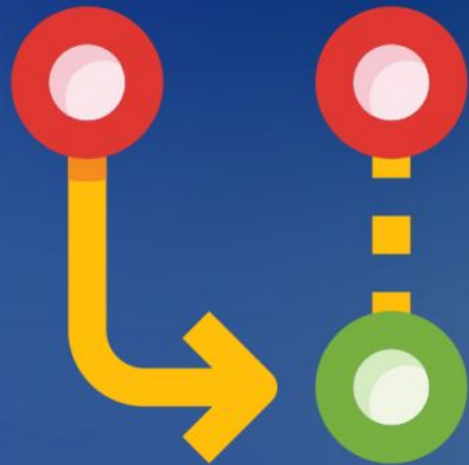


Cuando los commits están bien organizados, es más sencillo revisar el código.



¿Qué es un Commit?

Es una acción que registra los cambios realizados en los archivos de un proyecto en un repositorio. Esencialmente, un commit representa una instantánea del estado actual del código en un momento específico en el tiempo. Cada commit está acompañado de un mensaje descriptivo que resume los cambios realizados.



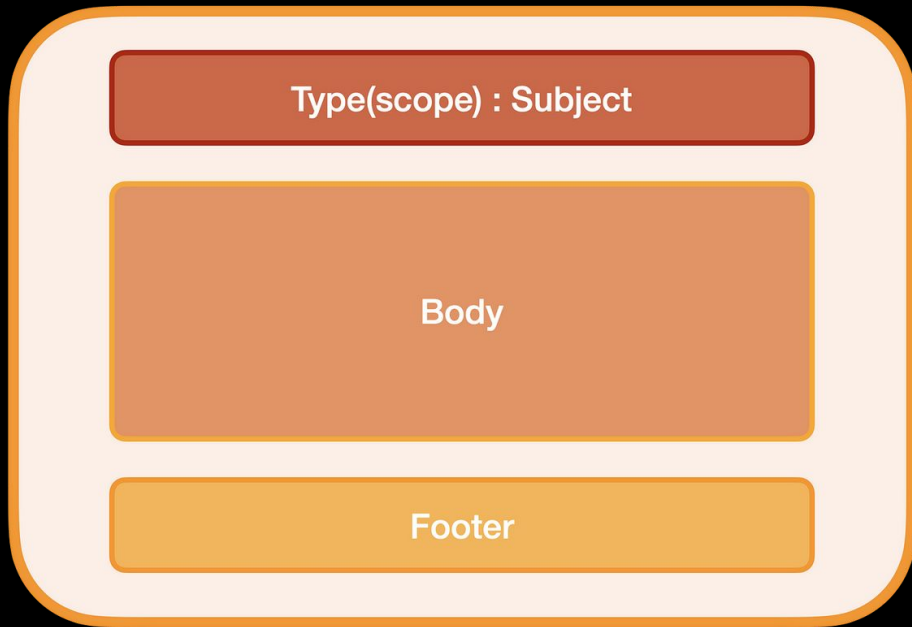


Estructura de un buen commit

Conventional Commits

Son un estándar de mensajes de commit que mejoran el control de versiones en el desarrollo de software. Con tipos de commit específicos y mensajes bien estructurados, proporcionan claridad, organización y automatización en la gestión de versiones, simplificando la colaboración entre los desarrolladores.

La estructura de un conventional commit se divide en 3 partes: **el título, el cuerpo y pie**



Type: Está contenido en el título y puede ser uno de estos tipos:

- **feat:** Una nueva característica.
- **fix:** Una corrección de errores.
- **docs:** Cambios en la documentación.
- **style:** Formato, faltan puntos y comas, etc.; sin cambio de código
- **refactor:** Refactorización del código de producción.
- **test:** Todo lo relacionado a pruebas.

Scope (opcional):

Se refiere a la parte del código o del proyecto a la que se aplica un cambio específico.

Subject:

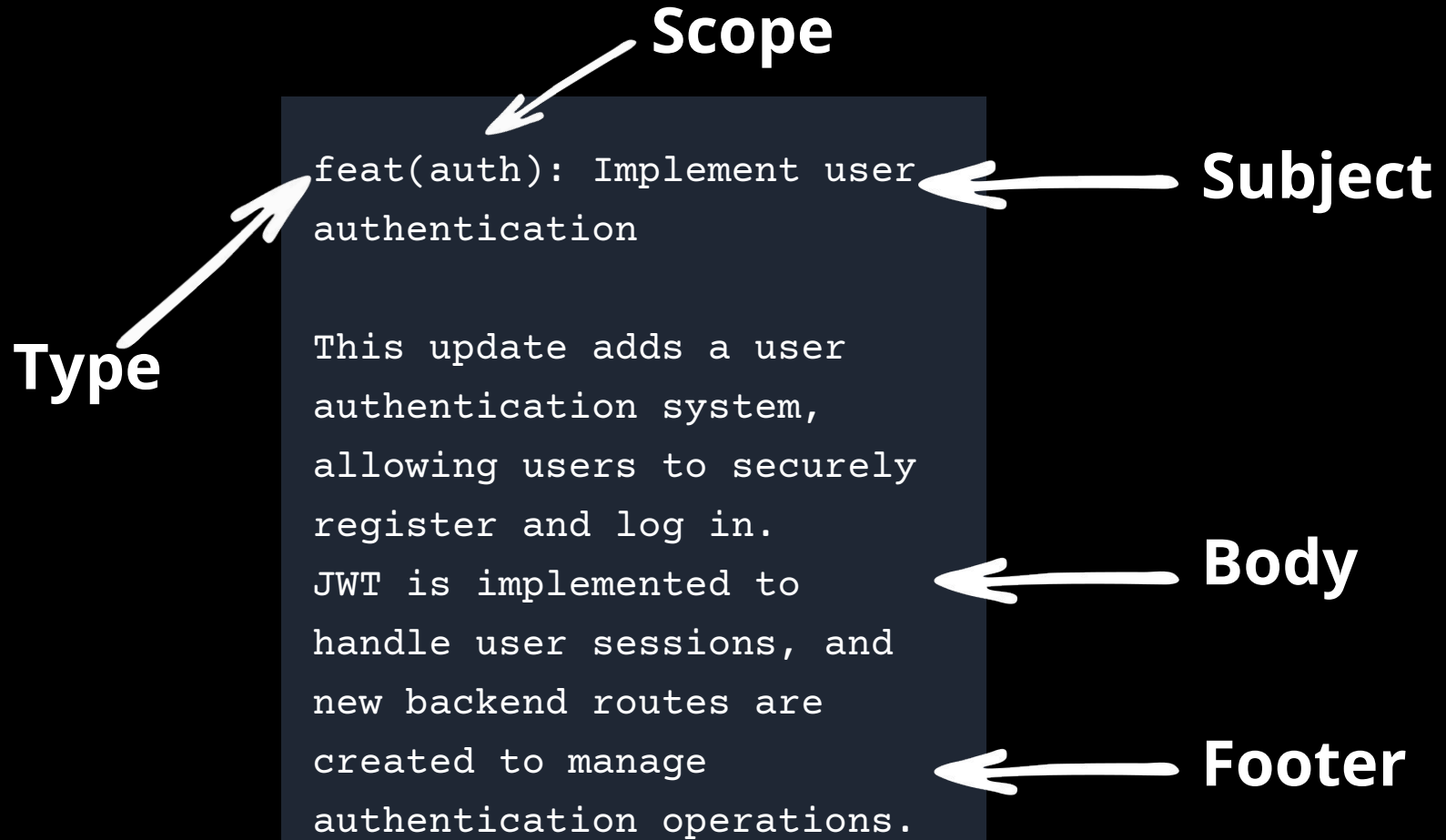
Aquí se debe explicar de manera clara y concisa qué hace el commit.

Body(opcional):

Proporciona detalles en el cuerpo del commit si es necesario, se explica por qué se realizó el cambio y cómo afecta al código.

Footer(opcional):

Proporciona información adicional, como referencias a problemas, enlaces a documentación, etc



- * 7d0fc3e typo
- * 8fc509a more changes
- * efe5fc5 add test
- * 447c5a0 updates
- * 1189cf0 update condition
- * 68abca0 updates
- * 29f73ed more changes
- * cefaa18 add file

	COMMENT	DATE
○	CREATED MAIN LOOP & TIMING CONTROL	14 HOURS AGO
○	ENABLED CONFIG FILE PARSING	9 HOURS AGO
○	MISC BUGFIXES	5 HOURS AGO
○	CODE ADDITIONS/EDITS	4 HOURS AGO
○	MORE CODE	4 HOURS AGO
○	HERE HAVE CODE	4 HOURS AGO
○	AAAAAAA	3 HOURS AGO
○	ADKFJSLKDFJSDKLFJ	3 HOURS AGO
○	MY HANDS ARE TYPING WORDS	2 HOURS AGO
○	HAAAAAAAAAANDS	2 HOURS AGO



Comandos esenciales de git

1

git init: Inicializa un nuevo repositorio Git en el directorio actual.

2

git clone [repository]: Crea una copia local de un repositorio remoto

3

git status: Proporciona información sobre el estado actual del repositorio.

4

git add [file(s)]: Permite agregar cambios al área de preparación, antes de realizar un commit.

5

git commit -m [message]: Crea un nuevo commit, describiendo en el mensaje los cambios realizados.

6

git pull: Busca y fusiona los últimos cambios del repositorio remoto en la rama actual.

7

git push: Envía los commits locales al repositorio remoto, haciéndolas accesibles para otros colaboradores.

8

git checkout [branch]: Cambio hacia alguna rama del repositorio.

9

git merge [branch]: Combina cambios de una rama en otra, asegurando que todos los cambios se incorporen mientras se mantiene la integridad de la base de código.

10

git log:

Muestra el historial de commits del repositorio

11

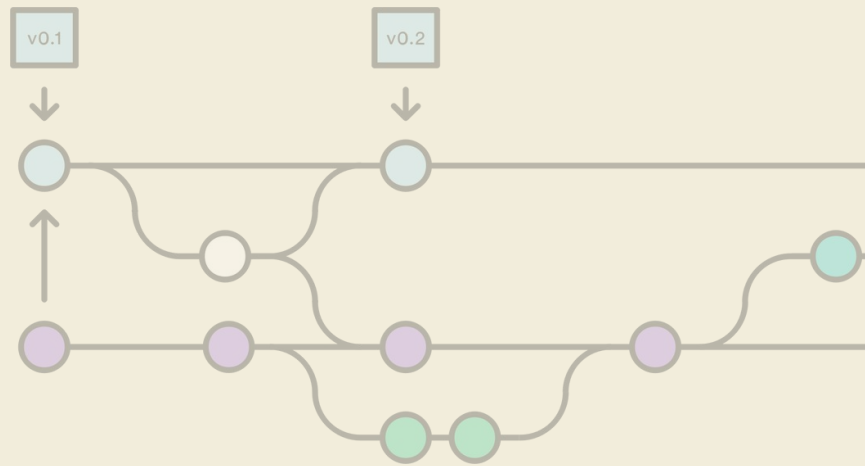
git branch:

Lista todas las ramas del repositorio, tambien se puede usar para crear, renombrar o eliminar ramas.

12

git tag [nombre] :

Crea una etiqueta (tag) apuntando a un commit específico, útil para marcar versiones de lanzamiento



GIT FLOW

Qué es Git Flow?

Es un modelo de flujo de trabajo para gestionar el desarrollo de software utilizando el sistema de control de versiones Git.

Se basa en la idea de tener ramas específicas para diferentes tipos de desarrollo, como características, versiones de lanzamiento y correcciones de errores, lo que facilita la colaboración y el seguimiento del progreso del proyecto de sistemas de control de versiones basados en Git

Ventajas

- **Organización clara:** GitFlow proporciona una estructura organizativa definida.
- **Responsabilidades separadas:** Facilita la asignación de responsabilidades a través de ramas específicas.
- **Control del ciclo de desarrollo:** Ofrece un flujo de trabajo definido que guía el ciclo de vida del desarrollo.
- **Facilita la colaboración:** Mejora la comunicación y la coordinación entre equipos.
- **Aplicación de Integración continua:** Se integra fácilmente con prácticas de integración y despliegue continuo.

Desventajas

- **Complejidad inicial:** Requiere tiempo y esfuerzo para entender completamente el flujo de trabajo.
- **Proliferación de ramas:** Puede llevar a una sobrecarga de ramas, especialmente en proyectos grandes.
- **Rigidez metodológica:** Su enfoque puede no adaptarse a todos los proyectos o equipos.
- **Sobrecarga de mantenimiento:** Puede requerir más tiempo y recursos para mantener el flujo de trabajo.

Existen dos distinciones principales en las ramas que define GitFlow:



Ramas principales

Main
Develop

La principal característica de las ramas principales es que solo existe una de cada tipo. El objetivo es que no se instancien y que no reciban código de forma directa a través de commit, siempre tienen que recibir código a través de ramas auxiliares.



Ramas auxiliares

Feature
Releas
Hotfix

Por otro lado las ramas auxiliares se puede instanciarse todas las veces que se consideren necesarias, aquí se deben realizar los commits.

MAIN BRANCH

(master/main)

- Es la rama principal y debe contener siempre la versión más estable del proyecto. Esta rama contiene solo el código que ha sido completamente probado y está listo para ser liberado a los usuarios finales.
- Al momento de fusionar el código con la rama main/master se debe de hacer desde la rama release o en dado caso hotfix.
- Cuando se realiza una versión estable del software, se etiqueta la confirmación en la rama main con un número de versión o una etiqueta específica.

DEVELOP

- Se crea a partir de la rama main
- Es el punto de integración para las nuevas características y cambios provenientes de las ramas feature.
- Al prepararse para un lanzamiento, una rama release se crea desde develop.
- No es necesariamente estable pero si funcional



FEATURE

- Nace de la rama develop y se crea para implementar nuevas funcionalidades.
- Después de que la funcionalidad está completa y ha sido probada, se fusiona la rama feature en la rama develop.
- Una vez que la rama feature ha sido fusionada en develop, generalmente se elimina.
- Se recomienda nombrarlas de la siguiente manera:
feature/NuevaFuncionalidad.



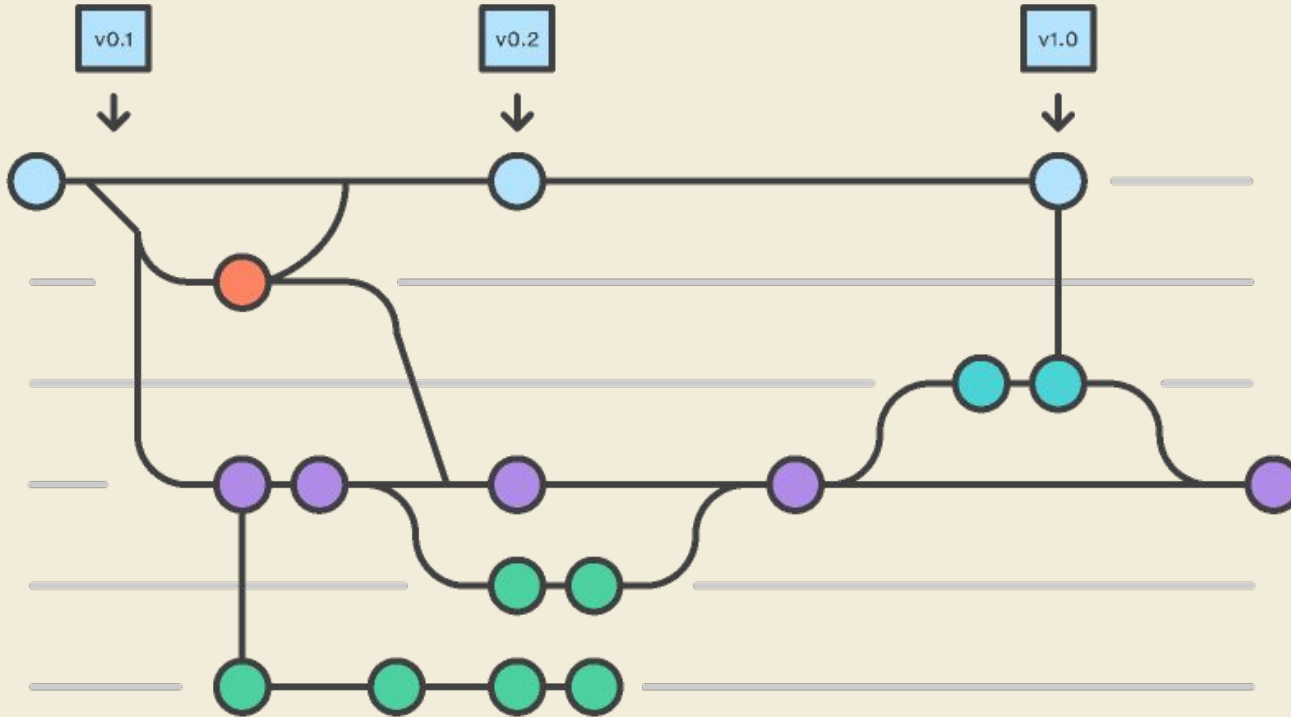
RELEASE

- Se crea para preparar y realizar lanzamientos de versiones estables del software.
- Al finalizar, se fusiona en la rama main y develop.
- Una vez que la rama release se ha fusionado en main y develop, generalmente se elimina para mantener un historial limpio.
- Se recomienda nombrarlas de la siguiente manera:
release/#Numero_version

HOTFIX

- Se utiliza para abordar y corregir problemas críticos en la producción de forma rápida y eficiente.
- Se crea directamente desde la rama main.
- Una vez que la corrección se ha implementado se debe fusionar en main y develop y a su vez se debe crear una etiqueta de versión específica.





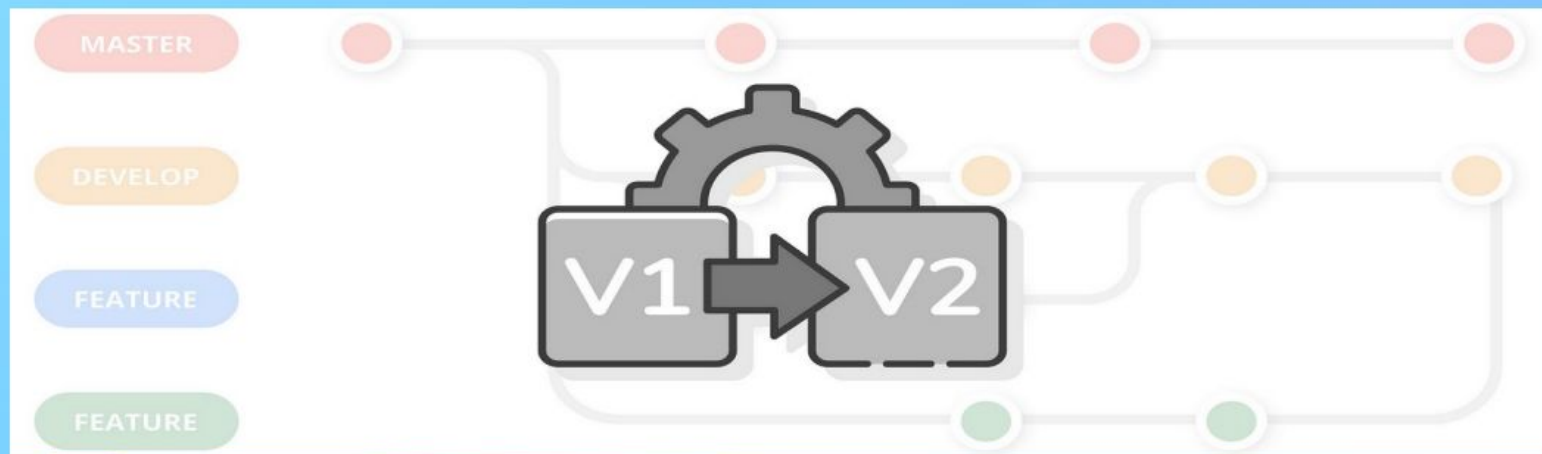
<https://www.atlassian.com/es/git/tutorials/comparing-workflows/gitflow-workfl>



Versionamiento Semantico

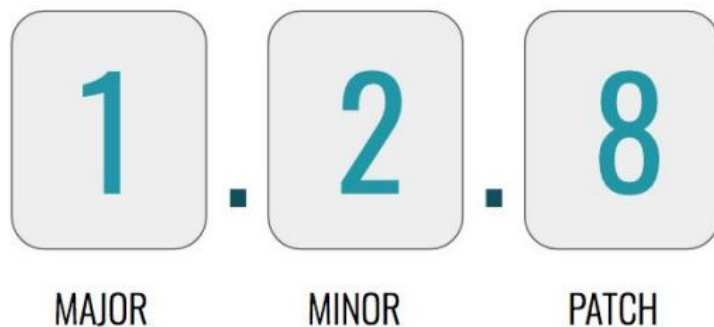
¿Qué es?

También conocido como SemVer (Semantic Versioning), es un esquema de numeración de versiones de software diseñado para transmitir el tipo de cambios que se han realizado en una nueva versión.



¿Cómo se Estructura?

La estructura del versionamiento semántico se expresa en el formato MAJOR.MINOR.PATCH:



VERSIONAMIENTO SEMÁNTICO

Patch

Incrementa cuando se realizan correcciones de errores compatibles hacia atrás. Esto incluye arreglos de bugs y mejoras menores que no añaden nuevas funcionalidades ni rompen la compatibilidad

Minor

Incrementa cuando se añaden funcionalidades nuevas de forma compatible con versiones anteriores. Los cambios realizados no rompen la compatibilidad con versiones previas, permitiendo que los usuarios adopten las nuevas características sin modificar su código existente.

Major

Incrementa cuando se realizan cambios incompatibles en la API. Esto significa que la nueva versión no es compatible hacia atrás y puede requerir modificaciones en el código que usa esta API

Importancia del Versionamiento Semántico

El versionamiento semántico proporciona un método claro y transparente para comunicar los cambios en el software, permitiendo a los desarrolladores:

- Comprender rápidamente la naturaleza de las actualizaciones.
- Gestionar dependencias de manera eficiente.
- Mantener la compatibilidad entre diferentes versiones del software.

Ejemplo

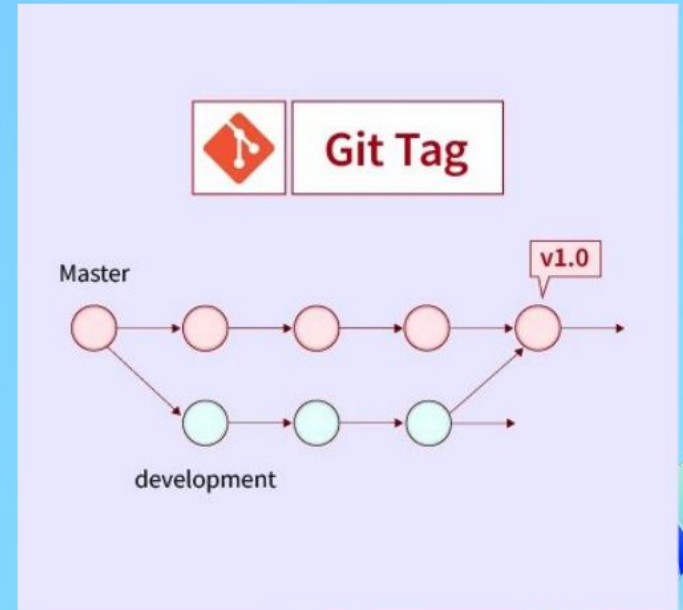
Si un software tiene la versión 2.1.3:

- El número 2 indica que el software va por la versión 2.
- El número 1 indica que es la primera actualización menor dentro de la versión mayor 2.
- El número 3 indica que es la tercera corrección de errores dentro de la versión menor 1 de la versión actual 2.



¿Qué es un Tag?

Es una referencia que apunta a un commit específico en el historial del repositorio, generalmente utilizado para marcar puntos importantes en el desarrollo, como versiones de lanzamiento. A diferencia de los branches, los tags no cambian su referencia al avanzar el desarrollo.



Git Tag

Master

v1.0

development

¿Cuándo generar un Tag?

- **Lanzamiento de Versiones:** Cuando se lanza una nueva versión del software (por ejemplo, v1.0.0, v2.1.3). Los tags permiten a los usuarios y desarrolladores regresar fácilmente a estas versiones.
- **Entrega de Funcionalidades Clave:** Al completar una funcionalidad importante o un conjunto de funcionalidades.
- **Finalización de Sprints/Iteraciones:** En metodologías ágiles, al concluir un sprint o iteración importante.

CONCEPTOS CLAVE APRENDIDOS

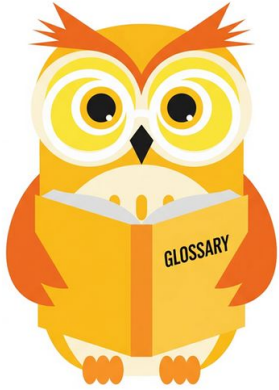
- GitFlow: como modelo de trabajo basado en la creacion de ramas en los repositorios, ayudan a llevar un orden entre versiones de software para evitar trabajar sobre codigo en produccion.
- Versionamiento Semantico: importante para poder saber que tipo de version de software estamos usando, si es la mas actual, se corrigio algun error o es una version compatible.
- Ramas: Se trabaja siempre sobre una base de ramas predefinidas, no obligatorias pero que ayudan a tener un orden, la main lleva solo codigo en produccion, develop sirve para trabajar todo el codigo antes del lanzamiento, hotfix corrige errores en produccion, realease permite lanzar una version funcional de la aplicacion.

REFERENCIAS

- <https://git-scm.com/book/es/v2/Inicio---Sobre-el-Control-de-Versiones-Acerca-del-Control-de-Versiones>
- <https://unity.com/es/topics/what-is-version-control>
- <https://nvie.com/posts/a-successful-git-branching-model/>
- <https://dev.to/imgildev/semver-que-es-y-por-que-es-importante-para-el-desarrollo-de-software-4ic1>
- <https://nvie.com/posts/a-successful-git-branching-model/>
- <https://gist.github.com/dasdo/9ff71c5c0efa037441b6>
- <https://git-scm.com/book/es/v2/Inicio---Sobre-el-Control-de-Versiones-Instalaci%C3%B3n-de-Git>
- <https://www.conventionalcommits.org/en/v1.0.0/>



Ejemplo práctico



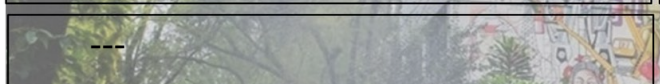
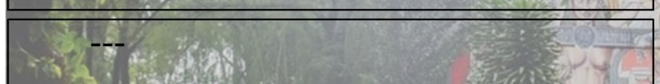
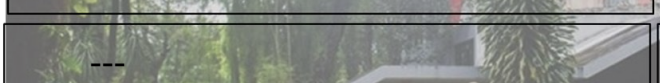
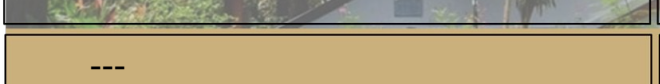
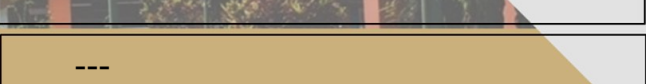
Practiquemos lo aprendido



Nombre de la actividad:	---
Cantidad de participantes:	---
Doy fe que esta actividad está planificada en dtt (Sí/No):	---

Hora de inicio:	---
Hora de fin:	---
Duración (min):	---

Participantes: llenar las siguientes cajas de texto (tomar información del chat del meet)

Recursos

- <https://www.youtube.com/watch?v=F8443XQafGU>
- Link del Quizz (cada instructor colocara su link)



DUDAS ¿?



**¡GRACIAS POR SU
ATENCIÓN!**

RECUERDA QUE TENEMOS NUESTRO FORO SEMANAL DONDE PUEDES CONSULTAR CUALQUIER DUDA
QUE TE SURJA EN LA SEMANA